

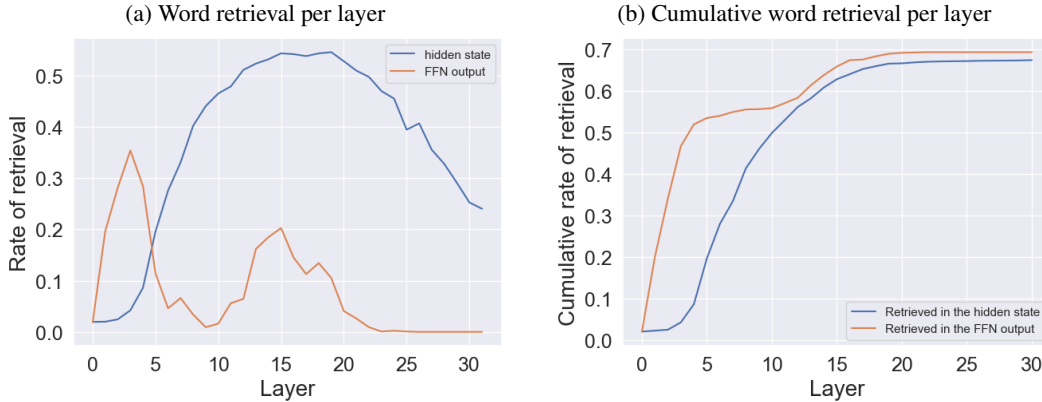


Figure 4: Word retrieval in FFN layers vs. in the hidden states. (4a) Retrieval rates across layers. The FFN values peak before word retrieval begins in the hidden layers. (4b) Its cumulative version, showing word retrieval occurs earlier and more frequently in FFNs than in the hidden representation.

5 HOW DOES DETOKENIZATION HAPPEN?

We have so far shown that LLMs perform a detokenization step: at some point, the hidden representation of the final token of a given word becomes strikingly similar to the (single-token $\times$) vector of that word. This process is robust to artificial splits of single token words, to splits due to typos, and even to multi-token words, which the model can still recognize at the input layer, despite never having seen them there during training.

We next turn to ask **how** does the model reconstruct full word representations from sub-word tokens? We aim to understand the dynamics of this process by analyzing the main transformer components: feedforward network layers (FFN) and the attention mechanism.

5.1 WORD RETRIEVAL USING THE FEEDFORWARD MECHANISM

FFN layers have been shown to serve as key-value tables for storing memory (Geva et al., 2022; 2021; Meng et al., 2022b). We hypothesize that this memory might be used to store the inner lexicon as well. Particularly, we suggest that the model uses the FFNs to refine the representation of the final token, enabling it to retrieve the original word.

To test this hypothesis, we repeat the typos experiment of single-token words (Sec. 4.1), but this time applying logit lens to the *output of the FFN layers*, instead of the hidden representation. Figure 4a shows the retrieval rate of the original word. We observe that the retrieval of the full word concepts from the FFN layers occurs a few layers earlier compared to the hidden state, which indicates that they help refine it. Nonetheless, the rate of retrieval appears to be lower in FFNs, which suggests that other factors might come into play in building the hidden representation of words. However, in Fig. 4b we plot the cumulative retrieval rate, i.e., for each layer, whether the word is identified in *any* layer so far. Our results indicate a different conclusion: the FFN update vectors match the (single-token) input representation of the word 70% of the time in any layer, particularly both earlier and more frequently compared to the hidden representation. This hints that FFNs indeed play a substantial role in building the internal word representations in LLMs.

We further investigate the role FFNs play in detokenization through ablation experiments on FFN updates in Llama2-7B. Specifically, we test whether these updates are necessary for the word representation to emerge in the residual stream. To do so, we artificially split single-token words in a similar setting to Sec. 4.1, this time focusing on derived words, formed by adding a suffix to a root word.¹⁰ We split these back to two parts, for example, we divide “eating” to “eat” and “ing”, so that processing the suffix token is essential to reconstructing the word correctly. Using logit lens, we ex-

¹⁰We examine words with three common suffixes: “ing”, “ion”, and “est”.

amine the FFN updates to the suffix token’s residual stream, and selectively ablate those associated with the original single-token word ($\sim 5\%$ of layers). As a control, for each word, we ablate an equal number of random FFN updates. Our results (Fig. 12 in App. E) show that removing the updates carrying full-word representations dramatically reduces retrieval rates—from 85% without ablation to just 18%. In contrast, ablating random FFN updates has little to no effect.

This indicates that FFN updates are essential for a detokenized representation to emerge in the final token. But does this affect the model’s ability to “understand” the word, particularly when predicting the next token? To investigate this, we evaluate the model’s ability to retrieve the capital cities of countries, using the prompt “The capital of [COUNTRY] is -----”. We use single-token country names artificially split into two tokens, and follow the previous ablation protocol. We observe notable patterns: randomly ablating FFN updates reduces performance from 88% to 74%,¹¹ likely due to disrupting the model’s factual recall mechanism (Meng et al., 2022b). However, specifically canceling updates detected as country’s name (similarly occurring in only $\sim 5\%$ of layers) causes a sharp drop to 41%. Overall, our results suggest that FFNs are central to detokenization, and actively reconstruct full-word meanings in LLMs.

5.2 TOKEN AGGREGATION

Our results suggest that LLMs use the FFN layers to store and retrieve word representations, which are accessed in the final token to reconstruct the complete word. However, this role of the FFN layers only begins to emerge around layers 3–4. **What happens earlier?** We build on previous work showing that early layers primarily integrate information from nearby tokens to compose entities (Lad et al., 2024), and hypothesize that the model starts by aggregating information from the previous sub-word tokens.

To test this, we extract all two-token words in a subset of 1,000 WIKITEXT-103 documents (a total of 5,571 words), and feed them, along with their context, to Llama2-7B. We then measure the average attention weights of the final token to the prefix token in each layer. As a control, we also measure the average attention weights assigned by single-token words to their preceding token.

Our results (Fig. 5) support previous findings (Lad et al., 2024)—the attention to previous tokens is high in the first 2–3 layers, but then declines sharply (by up to $\sim 90\%$).¹² For single-token words, we observe a similar attention pattern to the previous token in the first layers, but importantly—the initial peak is significantly lower than in multi-token words.¹³ Still, in later layers, the attention weights of single-token words are in turn *higher* than for multi-token words. These results suggest LLMs strongly attend to the preceding sub-word tokens of multi-token words *at first*, but then largely ignore them.

Altogether, our results suggest that LLMs perform a detokenization process by first aggregating information from the prefix tokens into the final token’s hidden representation, and then refining the representation of the final token using the FFN layers to retrieve the full word’s concept representation. This two-stage process of token aggregation and concept retrieval provides insight into the mechanisms LLMs use to handle sub-word tokens and reconstruct word-level representations.

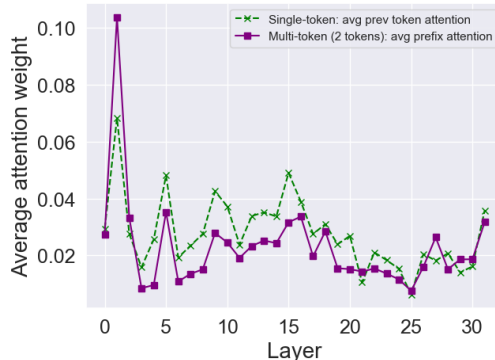


Figure 5: Attention weights for 2-token words: Early peaks (layers 2–3) show high attention values from the second sub-word token to the first, but these values decline rapidly. Attention from single-token words to their previous token shows a similar trend, though with substantially lower values at first, which become higher later.

¹¹We note that baseline performance when passing country names without artificial splits is 95%.

¹²Experiments with 3- and 4-token words show a similar trend, see App. B.

¹³The diverging patterns between single and multi-token words are statistically significant; see App. B.

6 EXPANDING LLM VOCABULARY WITHOUT FINETUNING

We have shown that language models internally fuse multi-token words into a single-token representation, and that they can further “read” these representations as inputs, and decode the original multi-token words. This raises the question: can models use these fused representations instead of the original multi-token inputs—to encode input prompts using less tokens and reduce computation? Similarly, models were shown to implicitly predict several future tokens in a single hidden state without being explicitly trained to do so (Pal et al., 2023); can we leverage these representations to enable models to predict multi-token words in a single inference step?

Motivated by our findings, we explore whether we can expand the model’s vocabulary with new input and output embeddings for originally multi-token words, without any updates to model parameters.¹⁴ This goal is of practical importance, especially for low-resource languages and domains: even tokenizers built for multilingual support often produce substantially longer token sequences for non-English languages—up to 13 times longer than equivalent English texts—impacting inference cost and speed (Ahia et al., 2023; Sengupta et al., 2023; Petrov et al., 2023). Still, prior attempts to expand tokenizer vocabulary post-hoc are limited in number, and require substantial additional training (Kim et al., 2024; Zhao et al., 2024). In contrast, we propose to detect words the model “knows” and successfully detokenizes to a single vector, and use this to obtain new token embeddings.

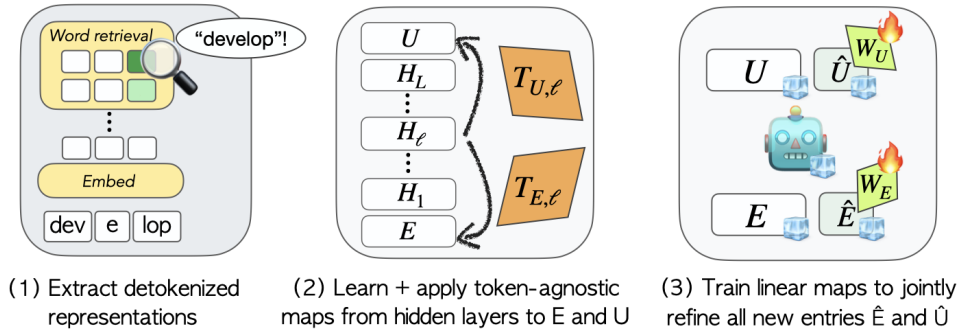


Figure 6: Our 3-step method to expand LLM vocabulary without updates to core model parameters.

Our framework for vocabulary expansion follows a 3-step process (Fig. 6). Given a multi-token word w that we would like to add to the vocabulary, and the model’s original input embedding and output unembedding matrices E and U with hidden dimension d , we (1) extract a detokenized, single-token representation r for w using a similar approach to Sec. 4.2: pass w_i as input to the model and apply PATCHSCOPEs (Ghandeharioun et al., 2024) with prompt P ¹⁵ to the last token’s hidden states at all layers. We then identify ℓ , the earliest layer at which the hidden state is successfully decoded into the full word, and set r as that hidden state. Next, we (2.1) learn a set of linear maps $T_{\ell,E}$ and $T_{\ell,U}$ to project hidden states from layer ℓ of the model to the embedding and unembedding spaces, based only on the *existing* in-vocabulary tokens.¹⁶ Then, (2.2) for each detokenized representation r taken from layer ℓ , we apply $T_{\ell,E}$ and $T_{\ell,U}$ to obtain the initial entries in the embedding and unembedding matrices to represent the word, $\hat{e}$ and $\hat{u}$. Finally, after computing the initial entries for all new words, we (3) refine the new representations to obtain the final entries: we initialize two $d \times d$ all-zeros matrices W_E and W_U , and set the new entries as $e = \hat{e} + W_E \hat{e}$ and $u = \hat{u} + W_U \hat{u}$. We train the refinement matrices W_E and W_U jointly in a short continued pretraining run, while keeping all other parameters frozen. Finally, we compute the final e and u , and use these to represent the new word in the expanded vocabulary. Importantly, if a word is never successfully decoded from the hidden states in any of the layers in step (1), we assume it is not found in the model’s inner lexicon, and therefore do not add it to the vocabulary.

¹⁴We do add new entries to the model’s embedding and unembedding matrices, but these are intrinsically required to expand the vocabulary and represent the new vocabulary words.

¹⁵For P , we use the template “x x x” where x is the patched representation of the new word.

¹⁶We learn $T_{\ell,E}$ and $T_{\ell,U}$ by fitting an orthogonal procrustes transformation from the layer ℓ hidden states of all in-vocabulary tokens (when passed as inputs on their own), to their corresponding embeddings or unembeddings (see App. G for details). Importantly, these projections only rely on the model’s existing vocabulary, and do not depend on which multi-token words we choose to add to the vocabulary.